

Final Project Report

Emily Dunham & Cooper Maughan

Introduction

Methods

Data Analysis

```
library(ggplot2)
library(tidyverse)
library(refund)
library(gridExtra)
library(patchwork)

fires <- vroom::vroom("forestfires.csv")
```

Modeling

Regular Model

```
FullModel <- pffr(
  Y_log ~ ff(X_log, xind = s),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2, 1))
)
FullBetas <- coef(FullModel, n1 = 12, n2 = 12)$smterms[[2]]$value
```

using seWithMean for s(t.vec) .

```
R2_reg <- summary(FullModel)
# R-squared of .568
```

Historical Model

```
FullModel_Hist <- pffr(
  Y_log ~ ff(X_log, xind = s, limits = 's<t'),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2, 1))
)
FullHistBetas <- coef(FullModel_Hist, n1 = 12, n2 = 12)$smterms[[2]]$value
```

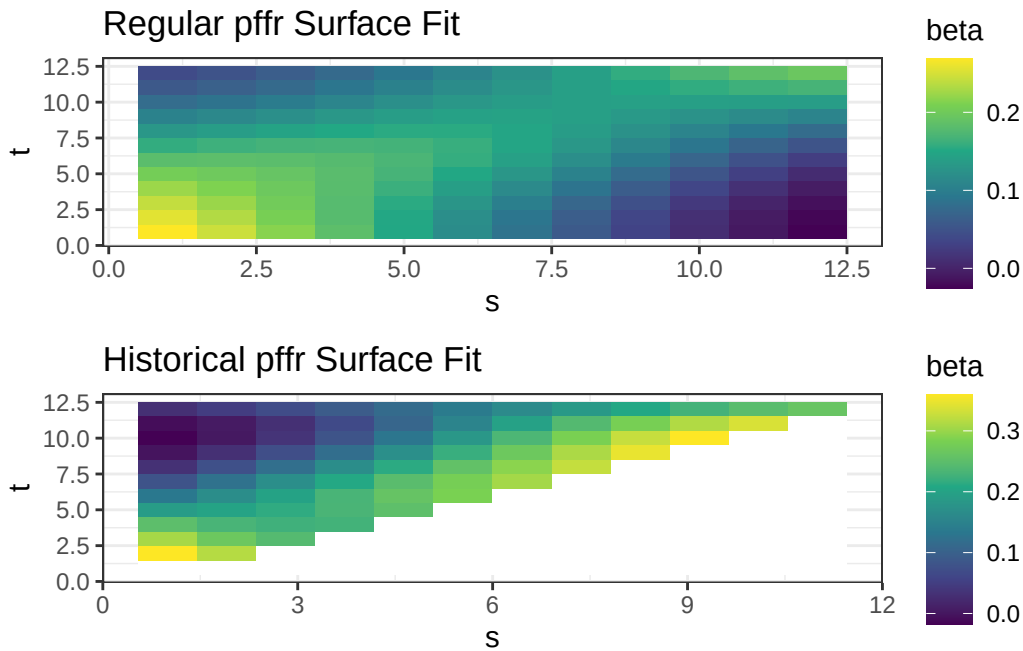
using seWithMean for s(t.vec) .

```
R2_hist <- summary(FullModel_Hist)
# R-squared of .6
```

Plotting the Models

using seWithMean for s(t.vec) .

using seWithMean for s(t.vec) .



Predictions

Regular Train Model

```
Train_Model <- pffr(
  Y_log_train ~ ff(X_log_train, xind = s),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2, 1))
)
Train_FullBetas <- coef(Train_Model, n1 = 12, n2 = 12)$smterms[[2]]$value
```

using `seWithMean` for `s(t.vec)` .

```
R2_full_train <- summary(Train_Model)
# R^2 = 0.613
```

Historical Train Model

```
Train_Model_Hist <- pffr(  
  Y_log_train ~ ff(X_log_train, xind = s, limits = 's<t'),  
  yind = t,  
  method = "NCV",  
  bs.int = list(bs = "ps", k = 5, m = c(2, 1))  
)  
Train_HistBetas <- coef(Train_Model_Hist, n1 = 12, n2 = 12)$smterms[[2]]$value
```

using `seWithMean` for `s(t.vec)` .

```
R2_hist_train <- summary(Train_Model_Hist)  
# R-squared of .637
```

Regular Model Prediction

```
X_log_test <- log(X_fires_test + 1)  
reg.train_preds <- predict(  
  Train_Model,  
  newdata = list(X_log_train = X_log_test)  
)  
# Look at RMSE  
y_test <- t(Y_area_test)  
reg.train_preds_t <- t(reg.train_preds)  
regular_rmse <- sqrt(mean((y_test - reg.train_preds_t)^2))  
regular_rmse
```

```
[1] 141.591
```

Historical Model Prediction

```
## We need to do this manually  
## Predict function doesn't work with limits  
# Get intercept  
smterms <- coef(Train_Model_Hist, n1 = 12, n2 = 12)$smterms[[1]]$value
```

```
using seWithMean for s(t.vec) .
```

```
pterm <- coef(Train_Model_Hist, n1 = 12, n2 = 12)$pterm[1, 1]
```

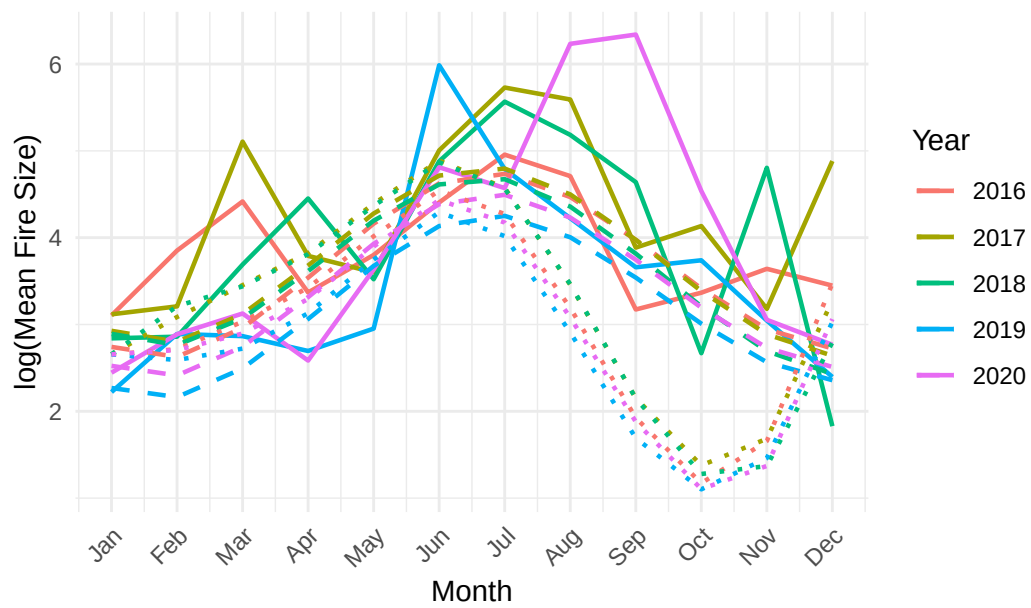
```
using seWithMean for s(t.vec) .
```

```
b0 <- pterm + smterm
b0_matrix <- matrix(b0, nrow = 12, ncol = 5)
# Get the beta surface matrix (12 x 12)
b1 <- Train_HistBetas
b1 <- matrix(b1, nrow = 12, ncol = 12)
b1[lower.tri(b1, diag = TRUE)] <- 0 # taking out values where s>=t
# Manual prediction
b1_matrix <- X_log_test %*% b1
b1_matrix_t <- t(b1_matrix)
hist.train_preds <- b0_matrix + b1_matrix_t
## Get RMSE's
historical_rmse <- sqrt(mean((y_test - hist.train_preds)^2))
historical_rmse
```

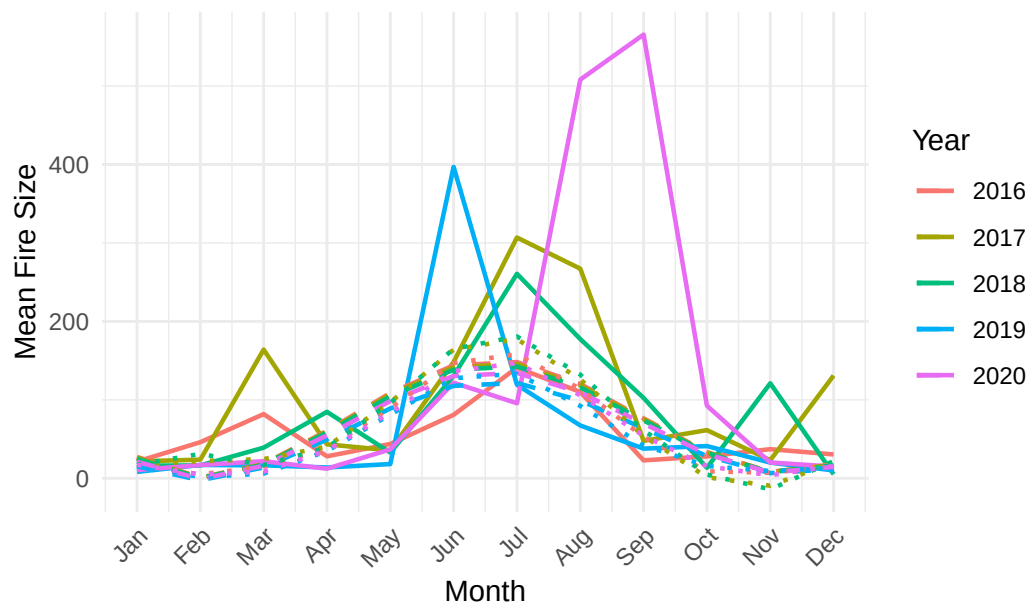
```
[1] 141.9687
```

Plot Predictions

Actual (solid) vs Regular (dashed) vs Historical (dotted) Predictions



Actual (solid) vs Regular (dashed) vs Historical (dotted)



Conclusion

What we found:

Our analysis suggests that yes, monthly fire counts do predict mean area burned, with both models explaining a meaningful portion of the variability. The regular model achieved an R^2 of 0.568 on the full data and 0.613 on the training data, while the historical model performed slightly better at 0.60 and 0.637 respectively. This is noteworthy because the historical model uses strictly less information (only past fire counts), yet predicts better. This suggests the historical constraint actually helps the model generalize rather than hurting it.

The beta surface plots tell the most interesting story. Early season fire counts (January–March) are the strongest positive predictors of area burned, showing up as bright yellow in the bottom-left corner of both surfaces. This effect decays as the year progresses. The historical surface also shows that the same-month and near-future relationships the regular model relies on are not as informative as the early season signal, reinforcing the early-season finding.

For predictions on the held-out 2016–2020 test years, both models achieved similar RMSEs (regular: 141.6, historical: 141.9), meaning neither substantially outperformed the other in actual forecasting despite the historical model fitting the training data better.

How certain we are:

We are moderately uncertain. We only have 29 years of data total, with just 5 test years, so the RMSE comparison is based on a very small test set and could easily flip with different train/test splits. The low effective rank warnings also suggest the fire count curves don't vary much in shape across years, which limits how much the functional predictor can really contribute beyond a scalar summary.

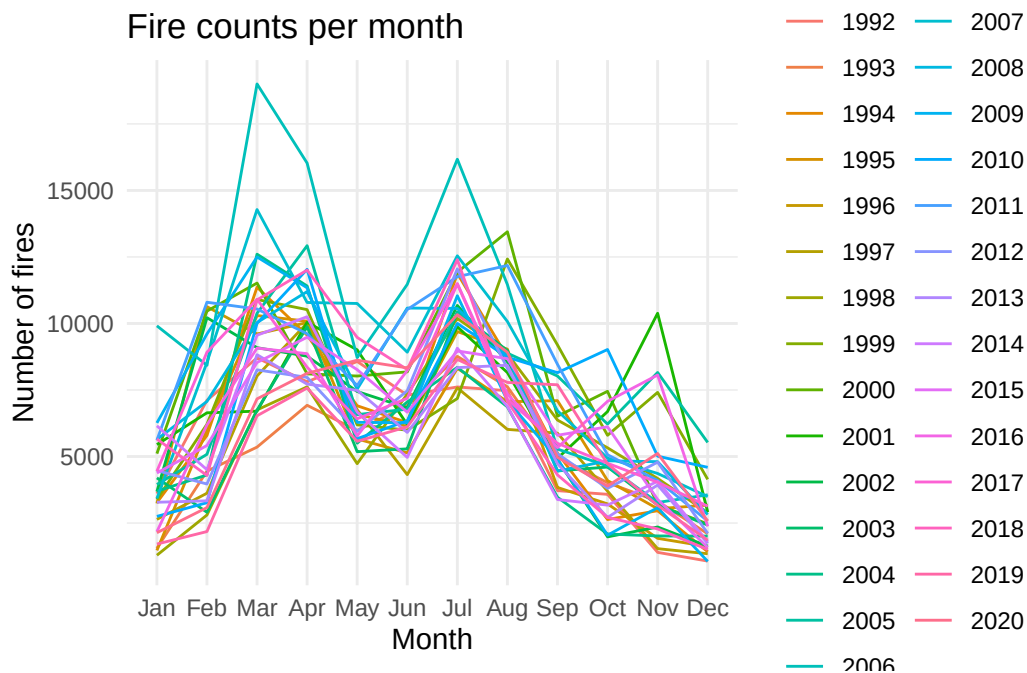
Follow-up questions worth exploring:

The most natural next question is whether adding climate variables like drought index, temperature, or precipitation as scalar predictors alongside the functional fire count predictor would improve predictions. Our plots show the model struggles most with extreme years like 2018 and 2020, which were anomalous fire seasons likely driven by climate conditions not captured in fire counts alone. We'd also ask whether predicting at a regional level rather than nationally would produce stronger relationships, since the national aggregation likely smooths out a lot of meaningful local variation.

Appendix

Exploratory Analysis

```
ggplot(  
  fires_counts,  
  aes(x = month, y = n_fires, color = factor(year), group = year)  
) +  
  geom_line() +  
  labs(  
    title = "Fire counts per month",  
    x = "Month",  
    y = "Number of fires",  
    color = "Year"  
  ) +  
  theme_minimal()
```



```
ggplot(  
  fires_month,  
  aes(x = month, y = total_size, color = factor(year), group = year)  
) +
```



```
geom_line() +
labs(
  title = "Total area burned per month",
  x = "Month",
  y = "Total FIRE_SIZE",
  color = "Year"
) +
theme_minimal()
```

